

AI QUALITY

0 Desafio Gilded Rose

Testes unitários gerados por IA sob auditoria: cinco técnicas de prompt na Iteração 1 e um prompt estruturado, guiado pelo grafo de fluxo de controle, na Iteração 2.

Go

ChatGPT

Gilded Rose Kata

CFG · $V(G) = 18$

Arthur Lara Panzera • Albert Luis • Diogo Caribe • Felipe Augusto Pereira • Gabriel Reis Lebron • Gustavo Azi Prehl

Setup do experimento

- Iteração 1: técnicas diferentes de prompt mudam a qualidade dos testes gerados por IA?
- Iteração 2: com acesso à estrutura interna (CFG), a IA percorre todos os caminhos ou só “arranha a superfície”?

Configuração

Linguagem	Go
LLM	ChatGPT
Kata base	Gilded Rose (Emily Bache)
Função analisada	UpdateQuality(items []*Item)
Repositório	github.com/gustavoprehl/TesteDeSoftware

Estrutura da equipe

- Dupla 1 · Engenharia de Prompt

Testar e documentar as técnicas de prompt
- Dupla 2 · Implementação em Go

Integrar testes, corrigir erros, capturar evidências
- Dupla 3 · Auditoria & Análise

Auditar cobertura, escrever veredito, montar slides

0 código legado - gildedrose.go

- **Função central:** UpdateQuality(items []*Item), um laço sobre os itens
- **Struct:** Item { Name string; SellIn, Quality int }
- **Tipos tratados:** Normal, Aged Brie, Sulfuras, Backstage Passes
- **Regras de limite:** Quality nunca < 0 nem > 50 (exceto Sulfuras, fixo em 80)
- **Estrutura:** 17 decisões aninhadas, uma condição composta (&&) e guardas repetidas

Ponto crítico (revelado na auditoria)

O fixture do kata cita “Conjured Mana Cake”, mas o código não implementa a regra de degradação dupla: não há nenhum if para esse tipo.

```

>>> 1/6 - Código legado (gildedrose/gildedrose.go) - a logica condicional confusa que a IA teve que testar. (sem verde/vermelho aqui - e so o contexto)
Pressione Enter para continuar...
package gildedrose

type Item struct {
    Name      string
    SellIn, Quality int
}

func UpdateQuality(items []*Item) {
    for i := 0; i < len(items); i++ {
        if items[i].Name != "Aged Brie" && items[i].Name != "Backstage passes to a TAFKAL80ETC concert" {
            if items[i].Quality > 0 {
                if items[i].Name != "Sulfuras, Hand of Ragnaros" {
                    items[i].Quality = items[i].Quality - 1
                }
            } else {
                if items[i].Quality < 50 {
                    items[i].Quality = items[i].Quality + 1
                    if items[i].Name == "Backstage passes to a TAFKAL80ETC concert" {
                        if items[i].SellIn < 11 {
                            if items[i].Quality < 50 {
                                items[i].Quality = items[i].Quality + 1
                            }
                        }
                    }
                    if items[i].SellIn < 6 {
                        if items[i].Quality < 50 {
                            items[i].Quality = items[i].Quality + 1
                        }
                    }
                }
            }
        }
        if items[i].Name != "Sulfuras, Hand of Ragnaros" {
            items[i].SellIn = items[i].SellIn - 1
        }
        if items[i].SellIn < 0 {
            if items[i].Name != "Aged Brie" {
                if items[i].Name != "Backstage passes to a TAFKAL80ETC concert" {
                    if items[i].Quality > 0 {
                        if items[i].Name != "Sulfuras, Hand of Ragnaros" {
                            items[i].Quality = items[i].Quality - 1
                        }
                    }
                } else {
                    items[i].Quality = items[i].Quality - items[i].Quality
                }
            } else {
                if items[i].Quality < 50 {
                    items[i].Quality = items[i].Quality + 1
                }
            }
        }
    }
}

```

Metodologia - 5 técnicas de prompt testadas

1

Direto

Colar o código e pedir a suíte diretamente

13 testes

2

Chain-of-Thought

Raciocínio passo a passo antes do código

27 testes

3

Persona Pattern

Papel de Senior QA Engineer, 15 anos

36 testes

4

Few-shot

Template AAA fornecido como exemplo

24 testes

5

Spec-driven (bônus)

Regras de negócio explícitas no preâmbulo

23 testes

Teste = um cenário executável. Chain-of-Thought e Persona agrupam seus testes em funções com t.Run; contamos os testes, não as funções.

Resultados e ranking

Rank	Técnica	Testes	Compilou?	Decisão	Observação
1º	Persona Pattern	36	Sim	100%	Sem alucinação; único com slice nil e vazio
2º	Spec-driven	23	Sim	100%	Afirmou que 3 testes falhariam – eles passam
3º	Few-shot	24	Não → corrigido	100%	Struct inventada e *testing.t
4º	Chain-of-Thought	27	Sim	94,1%	Faltam D15F e D17F
5º	Direto (baseline)	13	Sim	91,2%	Faltam D15F, D16F e D17F

Critério do ranking: cobertura de decisão medida (Iteração 2); no empate, compilar sem correção e não alucinar.

Ranking original, antes da medição: Persona > Chain-of-Thought > Spec-driven > Direto > Few-shot.

123

testes executados

0

FAIL (após a correção do Few-shot)

100%

cobertura de statement

O runner reporta 131: ele conta também as 8 funções que apenas agrupam testes (4 do Chain-of-Thought e 4 da Persona).

Estudo de caso: Few-shot - de RED a GREEN

Erro de sintaxe



*testing.t minúsculo não existe em Go (linha 140)

Erro estrutural



gr := GildedRose{Items: items} – struct inexistente no código real (11 erros “undefined”)

Corrigido



24 PASS após as duas correções – nenhuma assertão alterada

```
DONE 0 tests, 1 failure, 1 error in 0.000s
>>> 3/6 - RED #2: mesmo corrigindo o typo, ainda nao compila -> 'undefined: GildedRose'. A IA usou uma struct do exemplo do prompt que nao existe no codigo real (a assinatura real e UpdateQuality(items []*Item))
Pressione Enter para continuar...
FAIL _evidence/few_shot_typo_fixed_only

=== Failed
=== FAIL: _evidence/few_shot_typo_fixed_only (0.00s)
FAIL    github.com/emilybache/gildedrose-refactoring-kata/_evidence/few_shot_typo_fixed_only [build failed]

=== Errors
_evidence/few_shot_typo_fixed_only/few_shot_test.go:17:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:31:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:45:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:59:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:73:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:87:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:101:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:115:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:129:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:143:8: undefined: GildedRose
_evidence/few_shot_typo_fixed_only/few_shot_test.go:143:8: too many errors

DONE 0 tests, 1 failure, 11 errors in 0.000s
```

Na Iteração 2

O prompt estruturado descreve a API real e proíbe inventar outra. A suíte compilou de primeira, sem nenhuma adaptação.

Achado crítico - Conjured não foi testado

Nenhuma das 5 suítes menciona Conjured. A omissão também está na implementação: não há ramo para esse tipo.

Entrada	Esperado (spec)	Atual (código)
SellIn=3, Quality=6	SellIn=2, Quality=4	SellIn=2, Quality=5
SellIn=0, Quality=6	SellIn=-1, Quality=2	SellIn=-1, Quality=4
SellIn=0, Quality=3	SellIn=-1, Quality=0	SellIn=-1, Quality=1

Regra esperada: antes da venda, a qualidade cai 2/dia; depois, 4/dia; nunca abaixo de 0. Os 3 testes manuais de Conjured falham de propósito: são a evidência da lacuna.

Outros achados da auditoria

Oráculo derivado do código

A IA deduziu as regras do próprio código, que já omitia Conjured.

Sulfuras = 100 legitima estado inválido

Chain-of-Thought preserva Quality 100; a spec fixa Sulfuras em 80.

Faltam trajetórias de vários dias

Quase todos os testes fazem uma única chamada de UpdateQuality.

Item nil não testado

Persona testa slice nil e vazio, mas nenhuma suíte testa item nil na lista.

Veredito da Iteração 1

O falso positivo: 100% de cobertura de statement não significa 100% das regras de negócio testadas.

O que funcionou

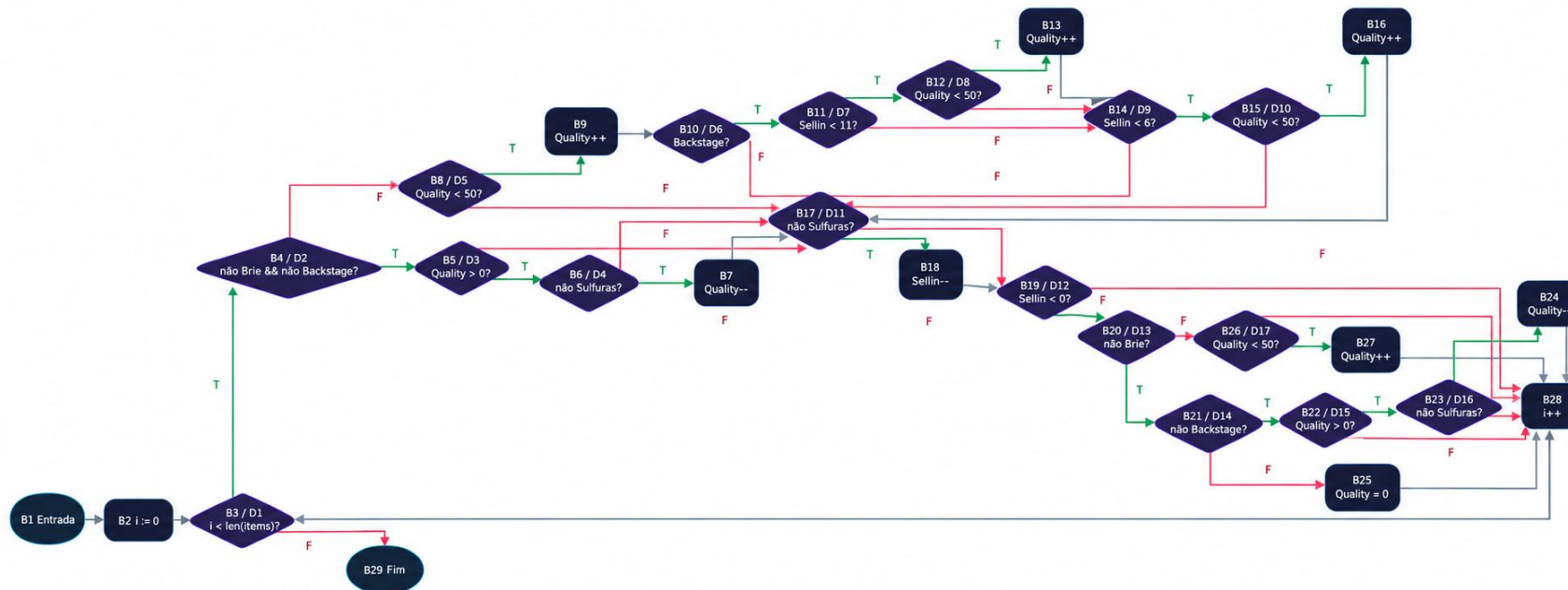
- ✓ 123 testes, 0 FAIL, 100% de cobertura de statement
- ✓ Persona, Spec-driven e Few-shot cobrem os 34 ramos
- ✓ Persona testa slice nil e vazio
- ✓ Vários tipos de item na mesma chamada

A falha crítica

- X Conjured não implementado nem testado
- X Direto e CoT deixam vencimentos sem teste: item comum com $Q = 0$, Sulfuras vencido, Brie com $Q = 50$
- X Few-shot e Spec-driven alucinaram
- X Oráculo derivado do código, não da especificação

Pergunta para a Iteração 2: statement coverage não separa as suítes. Precisamos de um critério que olhe os dois lados de cada decisão e de um prompt que dê essa estrutura à IA.

Grafo de Fluxo de Controle de UpdateQuality



29 nós
B1-B29

45 arestas
34 de decisão + 11 sequenciais

17 decisões
D1-D17

34 ramos
17 decisões × 2

Granularidade: cada expressão condicional completa é uma decisão; a condição composta da linha 11 (A && B) é um único nó (D2), com o curto-circuito documentado à parte.

Escopo: fluxo normal com itens válidos; o panic de um item nil não vira aresta.

Complexidade ciclomática e caminhos independentes

Pelos nós predicados

$V(G) = P + 1$

$17 + 1 = 18$

Por arestas e nós

$V(G) = E - N + 2$

$45 - 29 + 2 = 18$

As duas fórmulas convergem: $V(G) = 18$ caminhos linearmente independentes, o tamanho de uma base para cobertura de caminhos.

ID	Entrada	Saída	ID	Entrada	Saída
C1	lista vazia	–	C10	Backstage, 8, 49	7, 50
C2	Comum, 5, 10	4, 9	C11	Backstage, 3, 10	2, 13
C3	Comum, 5, 0	4, 0	C12	Backstage, 3, 49	2, 50
C4	Sulfuras, 5, 80	5, 80	C13	Comum, 0, 10	-1, 8
C5	Sulfuras, 5, 0	5, 0	C14	Comum, 0, 1	-1, 0
C6	Aged Brie, 5, 10	4, 11	C15	Sulfuras, -1, 80	-1, 80
C7	Aged Brie, 5, 50	4, 50	C16	Backstage, 0, 10	-1, 0
C8	Backstage, 15, 10	14, 11	C17	Aged Brie, 0, 10	-1, 12
C9	Backstage, 8, 10	7, 12	C18	Aged Brie, 0, 49	-1, 50

Entrada = (item, SellIn, Quality) · Saída = (SellIn, Quality)

Critério escolhido e proposição manual: 8 casos

Comandos
Toda linha executada
Iteração 1: 100% em todas as suítes.

Decisão (escolhido)
Cada decisão True e False: 34 ramos
Mínimo: 8 casos. É a linha do quadro final.

Caminhos
Base de V(G): 18 caminhos
18 casos. Mais forte e mais caro.

Caso	Item	Entrada (SellIn, Q)	Saída	Ramos que só ele cobre
M1	Comum	0, 10	-1, 8	D4T, D16T
M2	Comum	0, 0	-1, 0	D3F, D15F
M3	Sulfuras	-1, 80	-1, 80	D4F, D11F, D16F
M4	Aged Brie	0, 10	-1, 12	D6F, D17T
M5	Aged Brie	0, 50	-1, 50	D5F, D17F
M6	Backstage	15, 10	14, 11	D7F, D9F
M7	Backstage	3, 49	2, 50	D8F, D10F
M8	Backstage	0, 10	-1, 0	D8T, D10T, D14F

Por que 8 é o mínimo?

3

Comum + Sulfuras
D3F, D4T e D4F exigem três entradas diferentes

2

Aged Brie
D5F e D6F não cabem no mesmo caso

3

Backstage
D7F, D8F e D10T forçam três ingressos

3 + 2 + 3 = 8 · 34/34 ramos

Do prompt ingênuo ao prompt estruturado

Problema observado na Iteração 1	Elemento do prompt estruturado
A IA não sabia quais caminhos existiam; o Direto testou só o óbvio	Tabela das 17 decisões com linha, condição e destino True/False (o CFG em forma de tabela)
Nenhuma meta mensurável: “suíte completa” julgada no olho	Meta explícita: 100% de decisão, 34 ramos nomeados D1T...D17F, menor conjunto possível
CoT e Persona melhoraram porque raciocinavam antes do código	Processo obrigatório em 5 etapas: condições de alcance → rastreamento → matriz → minimalidade → código
Few-shot inventou GildedRose{} e *testing.t	API real descrita, proibição de inventar outra, pacote e import já definidos
Spec-driven afirmou falhas que não ocorrem	Todos os testes devem passar; a coluna ramos de cada teste vira afirmação verificável
Guardas de Quality leem valores já alterados	Observações estruturais sobre a ordem das mutações

O que o prompt não entrega, de propósito: nenhum valor de entrada, nem os casos M1-M8, nem as fronteiras difíceis, nem Conjured. Assim medimos se a IA melhora, não se ela copia.

Execução: ChatGPT, conversa nova, uma única tentativa, sem perguntas de acompanhamento.

Execução e validação da suíte estruturada

Não bastou passar: executamos, medimos cada ramo e conferimos cada afirmação da IA contra o código.

1 Compilação e testes

- Compilou de primeira, sem adaptação
- 8/8 testes PASS
- 100% de statement
- Nenhuma asserção alterada

2 Cobertura de decisão

- 34/34 ramos = 100%
- Medidor validado: M1-M8 dão 34/34
- Sem o M8, faltam exatamente D8T, D10T e D14F

3 Afirmações da IA

- Coluna ramos: 8/8 idênticas ao medido, teste a teste
- 34 condições de alcance × 7.644 entradas: 0 divergências
- Prova do mínimo de 8: correta

Veredito: sem alucinação. Na Iteração 1, com a mesma LLM, Few-shot inventou API e Spec-driven afirmou falhas inexistentes.

`go test ./prompt_tests/estruturado -v` • `go run ./_decisioncov -matriz ./prompt_tests/estruturado`

O que a IA gerou × o que o grafo exigia

Teste	Entrada	Saída	Caminho	Manual	Destaque
T1	Backstage, 0, 49	-1, 0	fora da base	M7 + M8	Q = 49: satura em 50 e zero
T2	Backstage, 5, 10	4, 13	C11	–	Os dois bônus sem vencer
T3	Backstage, 11, 10	10, 11	C8	M6	Fronteira exata SellIn = 11
T4	Aged Brie, 0, 0	-1, 2	C17	M4	Mesmo caminho
T5	Aged Brie, 0, 50	-1, 50	fora da base	M5 (igual)	Mesma fusão do grupo
T6	Sulfuras, -1, 10	-1, 10	C15	M3	Quality 10: inválido na spec
T7	Comum, 0, 2	-1, 0	C13	M1	Q = 2: menor valor para D15T
T8	Comum, 0, 0	-1, 0	fora da base	M2 (igual)	Mesma fusão do grupo

Caminho = caminho da base C1-C18 (slide 10) que o teste percorre.

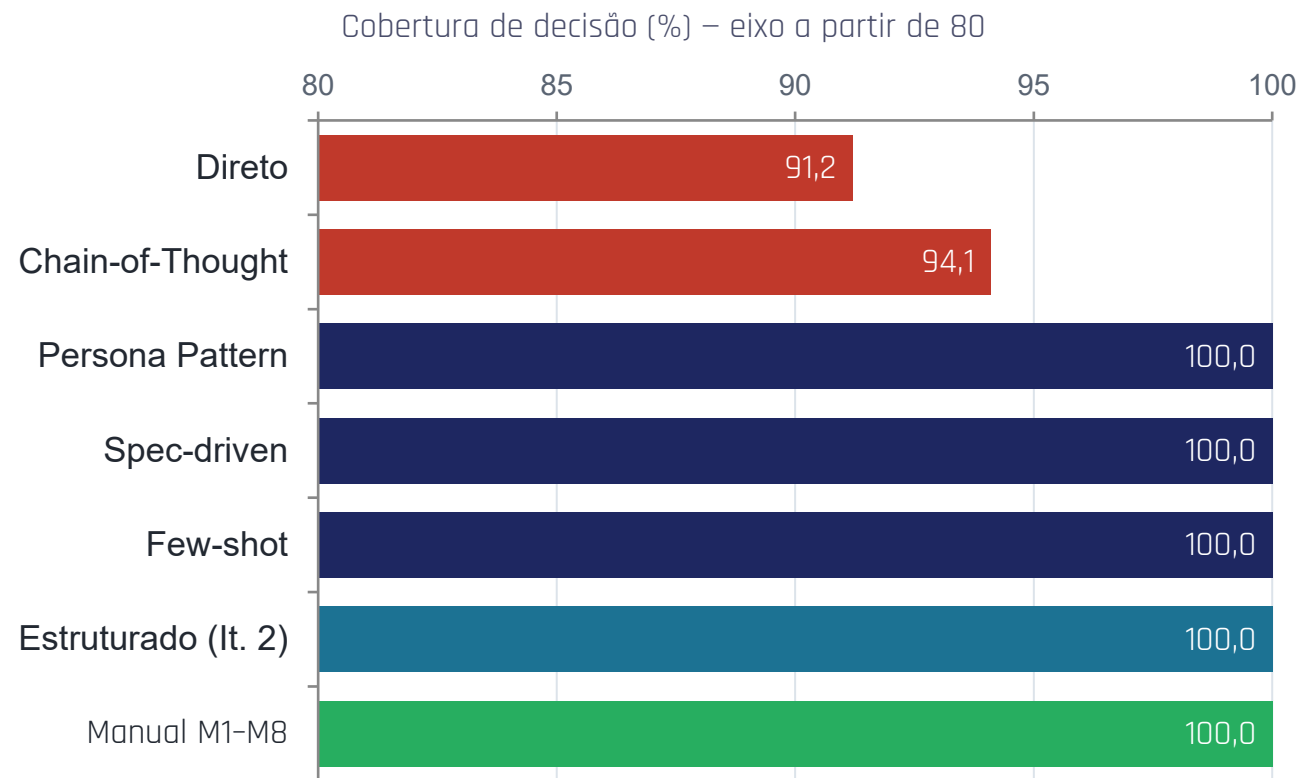
Acertos

- 8 testes = mínimo manual
- Achou Q = 49 e Brie com 50 sem o prompt dar valores
- Fronteiras SellIn 11 e 5

Onde ficou aquém

- 5 dos 18 caminhos da base (manual: 6)
- SellIn 10 e 6 não testados
- Sulfuras com Q = 10 vira “esperado”
- Nada sobre Conjured: não há ramo

Cobertura de decisão medida em todas as suítes



Suíte	Testes	Ramos	Não cobertos
Direto	13	31/34	D15F, D16F, D17F
Chain-of-Thought	27	32/34	D15F, D17F
Persona Pattern	36	34/34	–
Spec-driven	23	34/34	–
Few-shot	24	34/34	–
Estruturado (It. 2)	8	34/34	–
Manual	8	34/34	–

Leitura: os ramos que faltaram são vencimentos em fronteira – comum vencido com $Q \leq 1$ (D15F), Sulfuras vencido (D16F), Brie vencido com $Q \geq 49$ (D17F). O prompt estruturado cobriu os três com 8 testes.

Lacunas da Iteração 1: o que a Iteração 2 resolveu

Achado da Iteração 1	O que a Iteração 2 fez	Status
100% de statement escondia ramos sem teste	Cobertura de decisão medida em todas as suítes	Resolvido
Ranking subjetivo, sem meta mensurável	Meta de 34 ramos; ranking refeito pela medição	Resolvido
Alucinação de API (Few-shot)	API real no prompt; compilou de primeira	Resolvido
Alucinação de comportamento (Spec-driven)	Afirmações auditadas: 8/8 ramos, 0 divergências	Resolvido
Oráculo derivado do código	O prompt pede caracterização do código atual	Persiste
Sulfuras com qualidade inválida (CoT = 100)	T6 usa Sulfuras com Quality 10	Persiste
Conjured sem implementação nem teste	Sem ramo no código, fica fora do CFG; testes manuais seguem vermelhos	Persiste
Só uma chamada de UpdateQuality	Critério definido como 1 item em 1 chamada	Persiste
Item nil em lista não vazia	CFG modela só o fluxo normal	Fora do escopo

A estrutura resolveu o que era de estrutura: medição, meta e alucinação. O que depende da especificação continua precisando de auditoria humana.

Quadro comparativo final

Critério	Iteração 1 (Prompt Ingênuo)	Abordagem Manual (Teórica)	Iteração 2 (Prompt Estruturado)
Cobertura de Decisão	91,2% detectada 31/34 ramos	100% mapeada 34/34 ramos	100% detectada 34/34 ramos
Casos de Teste	13 gerados acima do necessário	8 necessários mínimo provado	8 gerados igual ao mínimo
Alucinações	Não no Direto*	N/A	Não auditado ramo a ramo

* **Prompt Ingênuo = prompt Direto (baseline).** Entre as outras técnicas da Iteração 1, houve alucinação no Few-shot (API inventada) e no Spec-driven (falhas inexistentes). Mesma LLM nas duas iterações: ChatGPT.

Veredito final consolidado

Iteração 1 – Direto

13 testes

31/34 ramos · 91,2%

Manual teórico

8 casos mínimos

34/34 ramos · 100%

Iteração 2 – Estruturado

8 testes

34/34 ramos · 100%

Mensagem final

Com a mesma LLM, o prompt estruturado levou a IA ao mínimo teórico: 34/34 ramos com 8 testes, rastreáveis e sem alucinação. Mas 100% de decisão não é 100% de caminhos (5 de 18) nem 100% dos requisitos: Conjured continua fora do grafo porque falta no código.

Próximos passos

- Implementar Conjured: os testes vermelhos viram regressão
- Testes de vários dias
- Oráculo da especificação ao lado da caracterização

Cobertura mede o código exercitado; auditoria mede se o código certo foi exercitado.

Mostrando a execução

Comandos rodados a partir de gilded-rose-go/, com a saída real resumida.

```
$ go test ./prompt_tests/estruturado -v
--- PASS: TestUpdateQualityCoberturaEstrutural/T1 ... T8      (8/8 PASS)

$ go run ./_decisioncov ./_decisioncov/validacao
_decisioncov/validacao      passam    34/34 (100.0%)

$ go run ./_decisioncov ./prompt_tests/...
SUÍTE      TESTES    DECISÃO      RAMOS NÃO COBERTOS
prompt_tests/chain_of_thought  passam    32/34 ( 94.1%)  D15F D17F
prompt_tests/direto            passam    31/34 ( 91.2%)  D15F D16F D17F
prompt_tests/estruturado       passam    34/34 (100.0%)
prompt_tests/few_shot          passam    34/34 (100.0%)
prompt_tests/persona_pattern    passam    34/34 (100.0%)
prompt_tests/spec_driven       passam    34/34 (100.0%)

$ go test ./manual_tests -v
--- FAIL: TestConjuredItemsDegradeTwiceAsFast  (3/3 FAIL)  ← lacuna de requisito, de propósito
```



Obrigado!

AI Quality – O Desafio Gilded Rose

github.com/gustavoprehl/TesteDeSoftware